Informe de Sprint 2

Antonio de Benito, Pablo Berástegui y Gonzalo García

Período: 31/01/2026 – 08/02/2026

Proyecto

TradeData – Análisis de Datos de Trading de Criptomonedas

1. Objetivo del Sprint

Al acabar el primer sprint, se logró almacenar nuestros datos sobre la criptomoneda `ETHUSD` en un bucket de S3. Sin embargo, al ser S3 un sistema de almacenamiento de objetos generales, no es el lugar ideal para acceder a nuestros datos tabulares; no es cómodo ni conveniente hacer queries SQL ni otras consultas.

Por tanto, tendremos que crear un **catálogo en AWS Glue**, donde éste sea el encargado de acceder al bucket de S3 mediante los **metadatos** que conozca del dato concreto al que queremos acceder (nombre, contenido, ruta, etc.). De esta manera, tenemos un intermediario que se encarga de acceder a nuestros datos y proporcionarlos de manera consistente siempre, sin que nosotros nos tengamos que preocupar de la ubicación de los mismos.

2. Historias de Usuario Incluidas

2.1. Historia de usuario 3

Como: Manager del proyecto
Quiero: Que las consultoras:

- Utilicen el servicio AWS Glue Data Catalog y AWS Glue Crawler para leer y guardar los metadatos relacionados con los datos históricos almacenados en S3.
- El nombre de la base datos debe tener la siguiente nomenclatura: `trade_data_<grupo>` (ejemplo: `trade_data_imat3a01`).
- El nombre de las tablas debe contener en su nombre referencias a la criptomoneda (ejemplo: `btc`).
- Se deberá de hacer un script en Python para automatizar la creación y ejecución a demanda del crawler.

Para: Poder empezar el proyecto con garantías

2.1.1. Análisis

Para poder empezar el proyecto con garantías necesitamos disponer de un catálogo de metadatos consultable sobre los datos históricos en S3. Para ello, analizamos cómo integrar AWS Glue Data Catalog y un AWS Glue Crawler que descubra el esquema de los ficheros y cree/actualice las tablas automáticamente, respetando la nomenclatura exigida (base de datos `trade_data_imat3a03` y tablas con referencia a la criptomoneda, p. ej. `btc`). Por último, se plantea un script en Python que permita crear y lanzar el crawler bajo demanda para mantener el catálogo sincronizado.

2.1.2. Implementación

Lo ideal para que la arquitectura sea escalable y consistente siempre es crear un script de generación del crawler y su tarea correspondiente. En nuestro caso, queremos que el crawler tenga permisos (*policies*) para acceder a S3 y a las herramientas básicas de AWS Glue. Adicionalmente, el crawler accederá a los csv que encuentre en nuestro bucket raíz del proyecto y los guardará en una misma tabla (todas las particiones anuales). De esta manera, podremos hacer consultas SQL desde Athena (editor nativo de AWS), para su posterior uso.

2.1.3. Evidencias

Aquí vemos el código usado para la creación del crawler.

```
import boto3

glue = boto3.client("glue", region_name="eu-south-2")
sts = boto3.client("sts")

account_id = sts.get_caller_identity()["Account"]
role_arn = f"arn:aws:iam::{account_id}:role/service-role/AWSGlueServiceRole-
    ethimat3a03"

response = glue.create_crawler(
    Name="crawler_eth_imat3a03",
    Role=role_arn,
    DatabaseName="trade_data_eth_imat3a03",
    Targets={
        "S3Targets": [
            {
                "Path": "s3://datos-grandes-eth-project/"
            }
        ]
    },
    Description="Crawler␣para␣datos␣ETH",
    TablePrefix="eth_",
    SchemaChangePolicy={
        "UpdateBehavior": "UPDATE_IN_DATABASE",
```

```
        "DeleteBehavior": "LOG"
    }
)

glue.start_crawler(Name="crawler_eth_imat3a03")
```

2.1.4. Resultado

datetime	symbol	open	high	low	close	volume	year
2022-01-24 01:00:00	BINANCE:ETHUSDT	2539.0	2539.75	2159.0	2439.29	1123134.70389	2022
2022-01-25 01:00:00	BINANCE:ETHUSDT	2439.56	2506.55	2348.96	2458.83	510152.954	2022
2022-01-26 01:00:00	BINANCE:ETHUSDT	2459.04	2722.0	2402.03	2462.99	806441.62289	2022
2026-01-01 01:00:00	BINANCE:ETHUSDT	2971.65	3010.0	2971.15	3004.19	136088.5207	2026
2022-01-27 01:00:00	BINANCE:ETHUSDT	2463.0	2521.0	2316.43	2424.25	599481.7159	2022
2022-01-28 01:00:00	BINANCE:ETHUSDT	2424.02	2558.98	2357.88	2544.99	485149.8065	2022
2022-01-29 01:00:00	BINANCE:ETHUSDT	2544.99	2636.66	2520.01	2601.43	376666.8246	2022
2022-01-30 01:00:00	BINANCE:ETHUSDT	2601.43	2640.0	2542.5	2601.68	290770.0859	2022
2022-01-31 01:00:00	BINANCE:ETHUSDT	2601.67	2706.77	2476.07	2686.94	444584.851	2022
2022-02-01 01:00:00	BINANCE:ETHUSDT	2686.94	2813.0	2673.52	2787.23	390034.27362	2022

Cuadro 1: Muestra del CSV del año 2022 descargado.

3. Arquitectura del Sprint

La arquitectura del sprint queda resumida en el siguiente flujo:

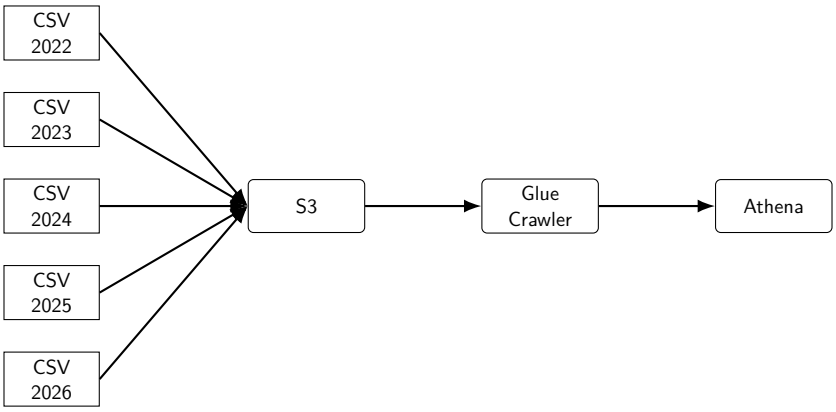

Figura 1: Arquitectura del sprint

4. Conclusiones del Sprint

La principal ventaja lograda al terminar este sprint es que hemos obtenido una vía automática para **extraer un catálogo de nuestros datos**; nosotros sabemos lo que queremos y AWS Glue sabe dónde se almacena. De esta manera, cualquier persona dentro de la empresa o de la consultoría será capaz de hacer consultas a los datos sin precisar de un conocimiento extenso en AWS S3 ni otras capas de la arquitectura.

5. Trabajo Pendiente y Próximo Sprint

De cara al siguiente sprint, el objetivo es abordar la HU-4 mediante los siguientes puntos:

- Implementar un almacenamiento de datos históricos basado en tres capas (separadas en carpetas distintas): Bronce, Plata y Oro.
- Usar AWS Glue y Apache Spark para transformar los CSV de la capa Bronce a Parquet en la capa Plata.
- Calcular indicadores técnicos a partir de los datos de la capa Plata y almacenar los resultados en la capa Oro.